

Chapitre 8.2: Atelier pratique : Requêtes et informations

1. ♦ Obtenir des informations sur les utilisateurs

Les commandes PowerShell permettent d'extraire rapidement des informations sur les utilisateurs avec différents niveaux de détail.

Découvrir les attributs disponibles

Avant de pouvoir rechercher des informations spécifiques, il est important de connaître les propriétés disponibles. Ces propriétés sont disponibles dans l'interface graphique via le menu **Propriétés > Editeur d'attributs** dans **Utilisateurs et groupes d'AD** (il faut d'abord activer l'option **Fonctionnalités avancées** dans le menu principal de **Utilisateurs et groupes d'AD**).

Commandes de base

```
# Lister tous les utilisateurs (limité aux 10 premiers pour l'exemple)
Get-ADUser -Filter * -ResultSetSize 10

# Obtenir un utilisateur spécifique par son SamAccountName
Get-ADUser -Filter {SamAccountName -eq "victor.vanhoof"}

# Obtenir les utilisateurs dont le nom commence par V (il faut que le nom
soit rempli!!)
Get-ADUser -Filter {Name -like "V*"}

# Obtenir un utilisateur avec des propriétés spécifiques
# La propriété 'Department' en PowerShell correspond au champ 'Service'
dans l'interface française d'AD
Get-ADUser -Filter {SamAccountName -eq "victor.vanhoof"} -Properties
DisplayName, EmailAddress, Department
```

Exemple pratique : Recherche avancée d'utilisateurs

Changez à la main le pays de quelques utilisateurs vers la Belgique.

```
# Trouver tous les utilisateurs de la Belgique
Get-ADUser -Filter {Country -eq "BE"} -Properties Country |
    Format-Table Name, SamAccountName, Country
```

Note: Format-Table permet de formater les résultats de manière lisible, c'est juste une option

Trouver les utilisateurs actifs

```
Get-ADUser -Filter {Enabled -eq $true} | Format-Table Name
```

Filtre avec AND : utilisateurs belges ET du service Ventes

```
# Note: Dans PowerShell, la propriété s'appelle 'Department' (en anglais)
# mais dans l'interface française d'AD, ce champ s'appelle 'Service'
Get-ADUser -Filter {Country -eq "B" -and Department -eq "Ventes"} -
Properties Country,Department |
    Format-Table Name, SamAccountName, Country, Department
```

Exercice : Trouvez tous les utilisateurs dont le nom contient "va" et affichez leur nom complet.

Exercice : Pensez à une recherche utile au niveau de votre Active Directory et exécutez-la.

2. ♦ Obtenir des informations sur les groupes

Les groupes sont essentiels dans AD pour gérer les permissions. PowerShell permet de les explorer facilement.

Commandes de base

```
# Lister tous les groupes
Get-ADGroup -Filter *

# Obtenir les membres d'un groupe
Get-ADGroup -Filter {Name -eq "GG-EU-RH-Users"} | ForEach-Object {
    Get-ADGroupMember -Identity $_.DistinguishedName
}
```

Rappel: ForEach-Object permet de parcourir chaque objet retourné par Get-ADGroup. Le \$_ représente l'objet actuel utilisé dans la boucle

```
# Obtenir les groupes d'un utilisateur
Get-ADUser -Filter {SamAccountName -eq "victor"} | ForEach-Object {
    Get-ADPrincipalGroupMembership -Identity $_.DistinguishedName
}
```

Exercice 1 : Trouvez tous les groupes dont le nom contient le mot "IT".

Exercice 2 : Affichez les membres du groupe "GG-EU-IT-Users". Utilisez Identity car Get-ADGroupMember n'accepte pas le paramètre -Filter.

```
# Solution: on utilise identity car Get-ADGroupMember n'accepte pas le
paramètre -Filter. Certaines commandes ne l'acceptent pas.
Get-ADGroupMember -Identity "GG-EU-IT-Users" | Format-Table Name,
SamAccountName
```

Le paramètre -Identity : Ce paramètre permet de spécifier quel objet AD vous voulez manipuler. Il accepte plusieurs formats d'identification. Par exemple... si on a le groupe "GG-EU-IT-Users", on peut le trouver de plusieurs manières en utilisant le paramètre -Identity :

- **Nom :** Simplement le nom du groupe (ex: -Identity "GG-EU-IT-Users")
- **SamAccountName :** L'identifiant unique du groupe dans le domaine (ex: -Identity "GG-EU-IT-Users")
- **DistinguishedName :** Le chemin complet dans l'AD (ex: -Identity "CN=GG-EU-IT-Users,OU=Groups,OU=EU,DC=computerelectronics,DC=be")
- **GUID :** L'identifiant unique global (ex: -Identity "123e4567-e89b-12d3-a456-426614174000")

3. ♦ Explorer les Unités d'Organisation (OUs)

Les OUs structurent votre Active Directory. PowerShell permet de les explorer et de comprendre leur hiérarchie.

Commandes de base

```
# Lister toutes les OUs
Get-ADOrganizationalUnit -Filter *

# Lister une OU spécifique
Get-ADOrganizationalUnit -Filter { Name -eq "RH" }

# Trouver un user spécifique par son nom
Get-ADUser -Filter {Country -eq "BE"}

# Obtenir les objets dans une OU spécifique
# Remarque: SearchBase est nécessaire car il définit le point de départ de
la recherche
Get-ADObject -Filter * -SearchBase
"OU=Users,OU=Comptabilite,OU=EU,DC=computerelectronics,DC=be"

# Compter les utilisateurs dans une OU
# Remarque: SearchBase est nécessaire pour limiter la recherche à une OU
spécifique
(Get-ADUser -Filter * -SearchBase
"OU=Users,OU=Comptabilite,OU=EU,DC=computerelectronics,DC=be").Count
```

4. ♦ Exportation des données

L'exportation des données est essentielle pour le reporting et l'analyse.

Exportation vers CSV

Note: Select-Object permet de sélectionner les attributs que l'on souhaite exporter.

Différence entre le pipeline (|) et ForEach-Object

Dans PowerShell, il existe deux façons principales de traiter plusieurs objets :

1. **Le pipeline (|)** : Utilisé pour passer les résultats d'une commande à une autre. Idéal quand la commande suivante accepte des objets en entrée via le pipeline.

```
# Exemple avec pipeline
Get-ADUser -Filter {Country -eq "BE"} | Select-Object Name,
SamAccountName
```

2. **ForEach-Object** : Utilisé quand vous devez exécuter **un bloc de code plus complexe** pour chaque objet ou quand la commande suivante n'accepte pas d'entrée via le pipeline.

Voici un exemple:

```
# Exemple avec ForEach-Object
Get-ADGroup -Filter {Name -like "GG-*"} | ForEach-Object {
    # Bloc de code exécuté pour chaque groupe
    Get-ADGroupMember -Identity $_.Name
}
```

Explication de cet exemple :

1. `Get-ADGroup -Filter {Name -like "GG-*"}` : Cette commande recherche tous les groupes AD dont le nom commence par "GG-"
2. `| ForEach-Object { ... }` : Pour chaque groupe trouvé, on exécute le bloc de code entre accolades
3. `Get-ADGroupMember -Identity $_.Name` : Pour chaque groupe (\$_ représente le groupe actuel), on récupère la liste de ses membres

Pourquoi utiliser ForEach-Object ici ? Parce que `Get-ADGroupMember` n'accepte pas directement des objets du pipeline. Il faut donc traiter chaque groupe individuellement et extraire son nom pour le passer à la commande.

Explication de \$_ : Dans PowerShell, \$_ est une variable spéciale qui représente l'objet actuel dans le pipeline. Quand vous utilisez ForEach-Object, \$_ fait référence à chaque objet qui est traité un par un. Par exemple, dans `$_ .Name`, le \$_ représente un groupe AD et `.Name` accède à la propriété "Name" de ce groupe.

```
# Exemple d'exportation avec pipeline
# Exporter la liste des utilisateurs vers un fichier CSV
# Note: 'Department' (propriété PowerShell) = 'Service' (interface
```

```
française d'AD)
Get-ADUser -Filter * -Properties Department, Title, EmailAddress |
    Select-Object Name, SamAccountName, Department, Title, EmailAddress |
    Export-Csv -Path "C:\utilisateurs.csv" -NoTypeInfo -Encoding
UTF8
```

Exportation au format HTML (pour visualisation dans un navigateur)

```
# Exporter vers un fichier HTML (plus visuel qu'un CSV)
# Note: 'Department' (propriété PowerShell) = 'Service' (interface
française d'AD)
Get-ADUser -Filter * -Properties Department, Title |
    Select-Object Name, SamAccountName, Department, Title |
    ConvertTo-Html -Title "Liste des utilisateurs" -Property Name,
SamAccountName, Country |
    Out-File -FilePath "C:\utilisateurs.html" -Encoding UTF8

# Ouvrir le fichier HTML dans le navigateur par défaut
Invoke-Item "C:\utilisateurs.html"
```

Exercice : Exportez la liste de tous les groupes et leurs membres dans un fichier CSV.